Python for computational Problem Solving · Unit_03 · Unit 3: Functions, Functional Programming and Modules

DRAFT — UE26CS151A · Unit_03 · Unit 3: Functions, Functional Programming and Modules · Generators

Standalone video lesson · authored study notes with source citations

Hugging Face PESU_Materials source tree, official units.json catalogue, transcript sidecars, slide text/PDF/PPT sources and the selected First_Year_8enterprise archive; see the provenance datasets in the release ZIP. Generated 18 October 2026.

· How to use this pack

Read the topic explanation, worked examples, and recap. Evidence citations point to source transcript intervals and slide pages; full slide images remain in the source visual index. Topics include the applicable parts: overview, key terms, formulas, figures, flowcharts, important points, worked examples, past-paper questions and a recap. IDs are shared with the separate documents: E = formula, G = graph, F = flowchart, W = worked example. Each ID is numbered by topic, so E4.2 is the second formula of topic 4.

Source text	Verbatim or OCR-derived text, kept separate from editorial notes.
Candidate	Automated topic/source association requiring manual confirmation.
Low ASR confidence	Transcript segment requires audio verification.
DRAFT	Human review remains incomplete.

Contents

1. 1 · Generators
2. Appendix A · Symbol glossary
3. Appendix B · Sources, coverage and gaps
4. Appendix C · Revision and retrieval practice

M1

Generators

1 Generators

Study focus: Generators. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000086 · EV0000087

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

In today's class, we will understand the concepts of generators.

Key details

- Previous class previous class, we have learnt iTraders.
- In order to do this or implement our own iTraders, we had to use ITE.
- Okay and then followed by we were also forced to make use of a next method.
- I have to make use of next methods.
- and I have to keep track of the internal states.
- The reason is the moment I will end.
- All the values that are present in a particular iterable, I have to stop.
- Otherwise, what do I need to do?
- I need to explicitly raise a stop iteration in order to indicate that there are no more values that need to be returned.
- I have to implement a class, I have to make use of an iter, I have to make use of a method function, then take care of the count because the moment there are no more values to be returned I have to raise a stop iteration.
- we basically make use of a generators in order to build itrators in a simpler way.
- But all these things are automatically handled by the generators in.
- Let me write a simple function, define.
- I'll say it as what?
- Nothing more in this case.
- I'll just say it as N is equal to one.
- Let me write the program and then I'll explain it to you.
- This gets printed first.

- So this is the first statement that gets printed first.
- And I will use one new statement called as yield.
- And I'll say it as what yield one.
- That we want to implement generators, definitely.
- It is very easy because I'm going to be creating my function the way in which we have done now.
- Only difference is when I use my regular functions, I would use a statement called as return.
- Yield represents a generator, return represents a particular function.
- If you want.
- To create your own generators, then it should must and should contain at least one yield statement.
- If you find an yield statement, then it is easily noticeable that it is your generator function.
- Yield as well as return gives you some value from a function.
- The only difference is the moment you come across with a return statement.
- You're talking about termination of the function entirely.
- You come out of the function, okay, exit out of the function completely.
- But with respect to your yields, you're pausing the function.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000086 · EV0000087 · EV0000088 · +1 source units in pack.json

Core explanation · 05:00–09:59

You're going to be saving all its state and then continue its execution.

Key details

- Return is basically for termination.
- is to store internal storage, internally store the state where you were basically before.
- Let me change my statement and say second.
- Do this gives so.
- I'll say it as last.
- This gets printed last and then followed by eight n.
- I can do this.
- Print next of A.
- Ready with this.
- Python generators.py.
- See, this gets printed first and first statement is printed.
- One and two needs to be printed.
- pauses the function, saves its states, and later continues from where.
- there is a successive call continues from there to the successive calls.
- I will s the moment you come across with the yield function here, the state saving everything will happen.
- Till some point until it come across comes across us with the next yield statement.
- The moment you come across with the next yield, it returns a value for you.
- I'll start from here, go till here.

- Again I have an yield statement.
- You have to stop at this point.
- Okay, but now what is the point of me using a next anyway, fine?
- A is your generator object, and I'll say.
- A print A.
- For loop, right?
- I'm sorry.
- Okay, see here, this gets printed first, this gets printed second, this gets printed last.
- Okay, and I did not use any iter method or next method, but still we are able to create our own iterators using generators.
- Please try to understand the concept of classes and then come back and understand the concept of I traders.

Example or application from the lecture

- And then let's call our function a is equal to my generator.
- Suppose if I do this.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000090 · EV0000091 · EV0000092 · +1 source units in pack.json

A Symbol glossary

Symbol	Name	Meaning	Appears in

B Sources, coverage and gaps

Source	Path	Use
Generators transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/012_Generators__c5a09f63b1677476.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Generators transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/012_Generators__c5a09f63b1677476.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Generators transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/012_Generators__c5a09f63b1677476.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9996; only the JSON segmentation supplies the displayed timestamps.
Generators transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/012_Generators__c5a09f63b1677476.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Historical question-source excerpts are shown only when a page has a strong text-based unit match. They are source references, not new solutions or exam predictions.

Known gaps, conflicts and corrections

1. 1 video-to-slide associations were not promoted to a match. Candidate rankings are listed in the transcript corpus and video-slide-matches.csv; the slide decks appear separately.
2. 4 transcript segments have low ASR confidence; they are flagged with timestamps in the transcript blocks and indexed in the corpus.
3. This is a visibly labelled source-preservation draft. Human page-level semantic and visual review is not complete; no unsupported subject summary or formula correction is claimed.
4. The 19.26 GiB MP4 files remain hosted on Hugging Face; the full video path/hash and transcript coverage are indexed. The full audio/video was not watched end-to-end.
5. Formula candidates are machine-extracted text with source locators; use the adjacent original slide page and the formula index before treating a candidate as an authoritative equation.
6. The study PDF contains edited, topic-organized lecture notes and textual slide-page takeaways. Full-page slide images remain in the source/visual index datasets, not embedded as the lesson. Human content, ASR, formula, and page-level visual review remains pending.

Coverage

Evidence is summarized by source to keep the reading copy legible. Full source-unit IDs, locators and decisions remain in pack.json.

Source	Unit counts	Example decisions
SRC00030	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare...
SRC00031	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare...
SRC00032	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00033	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare...

C Revision and retrieval practice